

1. Tìm 3 API public thật viết về một domain quan tâm, và trả lời: loại nào, base URL, auth method, có versioning hay không, resource identifier là số hay UUID?
 - GitHub REST API:
 - + Loại: REST.
 - + Base URL: <https://api.github.com/>
 - + Authentication Method: Sử dụng authentication token (thông qua personal access token – PAT, GitHub App, hoặc sử dụng GITHUB-TOKEN trong GitHub Actions).
 - + Versioning: Có version (hiện tại là 2026-03-10).
 - + Resource Identifier: Số, độc nhất trong một repository/scope nhưng không phổ biến trên toàn GitHub.
 - Roblox Open Cloud REST API:
 - + Loại: REST.
 - + Base URL: <https://apis.roblox.com/>
 - + Authentication Method: API keys và OAuth 2.0
 - + Versioning: Có version (hiện tại là v2)
 - + Resource Identifier: Số.
 - Spotify Web API:
 - + Loại: REST.
 - + Base URL: <https://api.spotify.com/>
 - + Authentication Method: OAuth 2.0
 - + Versioning: Có version.
 - + Resources Identifier: UUID.
2. Đọc Higginbotham Ch.1 + Geewax Ch.1. Tập trung định nghĩa, resource, đặc điểm của API tốt, so sánh với đặc điểm đã học và có điểm nào được bổ sung.
 - + Operational: Hệ thống sử dụng API nào thì cuối cùng cũng phải vận hành được và đạt được chức năng vốn có trong môi trường hoạt động.
 - + Expressive: API phải giúp người dùng truyền đạt lại được với hệ thống họ muốn cái gì rõ ràng và dễ hiểu.
 - + Simple: Các chức năng của API nên tránh phức tạp hóa một cách thừa thãi, nên đơn giản nhưng vẫn đủ chức năng để thực hiện nhiệm vụ
 - + Predictable: API cần phải nhất quán theo thời gian, qua các endpoints và versions, giúp cho người dùng sử dụng lại kinh nghiệm đã có và tăng hiệu năng.
 - + Documented: API cần phải được ghi chép đầy đủ các chức năng, và có ví dụ minh họa để hỗ trợ hình dung usecase.
3. Hoàn thiện Bài 6 và mở rộng.

```

@app.route("/books", methods=["GET"])
def book_list_route():
    filtered_books = [b for b in books_data.values()]
    q = str(request.args.get("q", "").strip().lower())
    sorter = str(request.args.get("sort", "").strip().lower())
    min_year = int(request.args.get("year", 0))
    if not q and not sorter and min_year == 0: # No querying
        b_list = books_data.values()
        return jsonify(list(b_list)), 200
    if q:
        filtered_books = [b for b in filtered_books if q in b["name"].lower().split()]
    if sorter:
        filtered_books = sorted(filtered_books, key=lambda b: b["name"])
    if min_year:
        filtered_books = [b for b in filtered_books if b["year"] >= min_year]
    return jsonify(filtered_books), 200

```